KorConcrete 0.134.4

Where Griffin Dow's concrete design app sits on KOR's drawing-to-model tools, a worked example, and a full security review. **Well built. Two things to fix before the team uses it.**

SUMMARY

Sound, with two small fixes before rollout

The update chain, the local engine's protection and the installer all hold up, including under live attack on a test install.

1. Lock the install folder and sign the installer

Today 23 staff accounts can replace the installer, and the installer is what carries your update-signing key.

2. Keep it off KOR-RDS01 for now

Until the app's front page stops serving the token, one user on a shared server could drive another user's engine.

Everything else is either sound or minor housekeeping, listed below.

WHERE KORCONCRETE SITS

On top of Ian's tools: drawings in, designed members out

Ian's tools cover the steps *before* KorConcrete: getting from the drawings to an ETABS model, and moving design forces into S-Concrete files without retyping them. KorConcrete does the design itself. Together they form one chain, and the one real test so far shows the hand-off between them working, apart from units.

STEP	WHAT HAPPENS	WHOSE TOOL
1. Drawings in	A PDF of the architect's plans, marked up by the engineer or not, goes in. Storeys, walls, columns, slabs and openings are read off the drawings, using KOR's own modelling rules held in the KorStandards database.	Ian's: `takeoff stickfile`
2. ETABS model out	An ETABS `.e2k` with every storey, wall and column placed, every wall labelled as a pier (`KPIER…`), a picture of every storey, and a written list of the assumptions it made, for the engineer to check.	Ian's
3. Analysis	The engineer opens the model in ETABS, checks it, applies loads and runs it, then exports the pier forces.	ETABS, the engineer
4. Design	KorConcrete reads the same `.e2k` and the pier forces and designs each wall and column to CSA A23.3-19.	**Griffin's: KorConcrete**
5. Paper trail	Both tools read and write S-Concrete `.SCO` files. Ian's writes the demands into an engineer's existing file without touching the section or bars, and builds one column schedule across a project's files.	Both

The example: Omar's hotel

Omar marked up the architect's PDF of 143 E 3rd Hotel by hand. Ian's tool turned that PDF, with nothing else, into an ETABS model of 41 storeys: 616 wall panels in 63 labelled piers, 384 column placements, and floors with their stair and shaft openings cut. Omar is reviewing the model now. Fed straight into KorConcrete, it read all 40 storeys that carry walls and our pier labels on each, so the hand-off works. The two unit faults it found are under *Testing with a real model*, at the end.

Where the two overlap, and the question for you

- **S-Concrete.** Ian's tool was built on the assumption that S-Concrete stays the checker, and it removes the typing around it. KorConcrete replaces the checker. If the team moves to KorConcrete, Ian's `.SCO` writer becomes a bridge for old projects rather than the main road.
- **Columns.** KorConcrete's `.e2k` import reads walls (piers), not frames, so columns do not reach it through the model today. Ian's model carries the columns. Reading them would complete the chain.

- **The engineer's method.** On 30961-01, Ian proved that the hand-made `.SCO` files match the engineer's Calculator sheet on 1,665 of 1,665 demands. Three inputs sit *before* that sheet: secondary loads added by hand, live-load reduction, and Rd/Ro scaling of the seismic forces. Reading KorConcrete's code, it builds the NBCC combinations from the ETABS pier forces, and it takes each of those inputs as a number the engineer types in. Live-load reduction is one factor per pier and storey, applied to the live load. Any ETABS case can be scaled by a factor per storey. Rd and Ro go into the chapter 21 ductility, shear-cap and overstrength checks. Its code makes no mention of secondary loads or tributary areas. **That is where the two ends join:** the model already knows the floors, the walls and the columns, so it is the natural place to work those numbers out instead of typing them.

SCOPE

What was reviewed, and how

Every part of the product was reviewed.

PART	HOW IT WAS REVIEWED
Installer (`Install KorConcrete.exe`)	Unpacked (Inno Setup 6.7.0); the setup script read in full; every installed file hashed and compared with the signed 0.134.4 update package
Launcher (`KorConcrete.ps1`, `Splash.cs`, `KorConcrete.vbs`)	Read in full
Splash program (`KorConcreteSplash.exe`)	Every string in the program traced back to `Splash.cs`
Engine (`KorConcrete-backend.exe`)	Unpacked; all 42 of your own modules disassembled and searched for risky calls (subprocess, pickle, eval, file and registry writes, network); the security module and every file-handling route read; all 38 routes listed
Bundled libraries	Version of every web-facing and file-parsing library read from the package
User interface	Both script bundles searched for outside calls and raw-HTML insertion
Live behaviour	Engine run on a test PC and attacked: missing and wrong tokens, forged Host headers, cross-site requests, path tricks, folder-name escapes
Install folder on FS01	Permissions on the folder and the installer read; group membership counted

FINDING 1 · FIX BEFORE ROLLOUT

Staff can write to the install folder

What: `\\Kor-fs01\Library\03 Programs\KorConcrete` gives *Full Control* to `BMZ_FS_Library_RW`. That group contains `BMZ_ROL_Staff`, which is **23 enabled user accounts**. `Install KorConcrete.exe` itself is unsigned.

Why it matters: your update chain trusts one public key, and that key arrives inside the installer. Anyone who can replace the installer can ship their own key, and from then on their own "signed" updates. The signature protects updates only as well as the folder protects the installer.

FIX

- Give write access to that folder to you and IT only. Everyone else reads. IT can make this change.
- Sign the installer with a code-signing certificate, so Windows shows *KOR Structural* as the publisher and refuses a tampered copy.

FINDING 2 · YOUR QUESTION

Terminal servers

What: the app's front page (`GET /`) hands the engine's token to anyone who asks for it. That is by design, so the app window can use it.

On a single-user PC this is harmless. Only that user's own programs can reach 127.0.0.1, and they already run as that user.

On a terminal server it is not. Every logged-in user shares the same 127.0.0.1. One user could read another user's token and drive their engine, including `/section/import-path`, which accepts a network (UNC) path. Pointing it at a network path makes the victim's session sign in to that address.

FIX

Have the launcher pass the token to the app window directly, for example in the address after a `#`, which the browser never sends to the server. The engine then never serves the token.

UNTIL THEN

Do not install it on KOR-RDS01. The answer to your question is yes: KOR has a terminal server, KOR-RDS01.

TESTED

What is sound

Update chain

- Each update carries an RSA-3072 signature over its version, SHA-256 and size.
- Size and hash are checked before anything is unpacked.
- File names inside the zip are checked before extraction.
- The app never downgrades, and it rolls back an update that fails to start.

Engine protection, tested live on 0.134.4

TEST	RESULT
Request with no token, or a wrong one	Refused, 401
Forged Host header (DNS-rebinding attack)	Refused, 421
Cross-site request from another website	No site is allowed. Your security layer runs first, so the browser never gets a readable answer
Path tricks to read files (`..` , `%2f` , `%5c`)	Refused; no file reached
Folder-name escapes in a design id (`..\..\` , `C:\`)	Refused by your name check; nothing written outside the data folder
Network listening	127.0.0.1 only; not reachable from other machines

Also confirmed: the token is 64 random hex characters, new on every start, and compared in constant time; security headers (CSP, no-sniff, no framing, no referrer) are on every response; exports come back to the window as data, and the engine never writes to a path it is given.

Installer

- It requests no admin rights and installs to the user's own profile (`%LOCALAPPDATA%\Programs\KorConcrete`).
- Its only registry change is the per-user `.korcol` file association.
- Its only run steps are opening the app after install and stopping the engine on uninstall.
- All 179 of 179 files it installs are byte-identical to the signed 0.134.4 update package. The splash program and the icon are the two additions, and the splash program matches its source.

Libraries: all current, and past their known security fixes

LIBRARY	VERSION	LIBRARY	VERSION
FastAPI	0.141.1	python-multipart	0.0.32
Starlette	1.6.0	pydantic	2.13.4
uvicorn	0.52.1	numpy	2.5.2
h11	0.16.0	openpyxl	3.1.5

User interface and other code

- Neither script bundle calls an outside address. The only web addresses in them are XML namespace names and React's error-help page.
- The code-clause navigator writes raw HTML only from text built into the app, and your CSP blocks any injected script regardless.
- `pickle` is used only to build a hash, never to load data.
- The only program the engine starts is a copy of itself, for the compiled-kernel self-check.

SMALL

Minor housekeeping

- **Development setting:** the engine still allows cross-site requests from your development server (`localhost:5173`). It does nothing today, because the token check runs first, but it should be removed from the packaged build.
- **Spreadsheet parsing:** openpyxl reads Excel files with Python's built-in XML parser, which does not fetch external entities. Adding the `defusedxml` package, which openpyxl uses automatically when present, hardens it further.
- **Signing passphrase:** your email says Ian also knows it; your security report says only you do. Worth settling which is true, and writing down who holds it and where the private key is backed up.

NOT SECURITY · FOUND WHILE TESTING

Testing with a real model

We fed an ETABS model built by Ian's drawing tools straight into your engine, through the same calls your window makes. The model was Omar's 143 E 3rd Hotel: 41 storeys, units LB/MM, walls carrying pier labels. Your engine read all 40 storeys that carry walls, and our pier labels on each of them. Two things go wrong with the units:

- **Lengths are read as feet.** A 3,239 × 305 mm wall on L19 (KPIER28) assembles as 987,177 × 92,903, which is 304.8 times too large. The `.e2k` importer assumes feet whatever the file's `UNITS` line says. Most KOR models are in inches, and the drawing-tool models are in millimetres.
- **Concrete strength.** The model's `FC 6.2000124` is in lb/mm², which is 27.6 MPa. The engine reads it as "6.2 MPa" and then sets it to 5 MPa.

WITH THOSE FIXED

The two tools connect end to end: a marked-up PDF becomes an ETABS model, the engineer runs the analysis, and KorConcrete designs the walls from the pier forces.